Fixing PDDL Domains Using Delta Debugging

Thomas Eckstein, Marco De Bortoli and **Gerald Steinbauer-Wagner**
Institute for Software Technology
Graz University of Technology, Graz, Austria
e-mail: steinbauer@ist.tugaz.at

Abstract

Developing planning domains can be a challenging task, because existing planners provide little feedback regarding the correctness of the domain and the reasons for failures in planning. In this paper we present an automated approach that can identify faulty preconditions in actions and show a working prototype implementation. Our debugger takes a faulty planning domain along with one or more problem instances as input. We use the delta debugging algorithm as the basis for our implementation, because it works by creating variations of the code in question and then testing it while essentially treating the interpreter, in our case the planner, as a black box. In our evaluation we introduce various errors into a set of selected domains taken from the International Planning Competition. We show that our debugger correctly identifies the mistakes in most cases and consistently outperforms a baseline based on brute force search.

1 Introduction

The field of AI planning [1] aims to solve planning problems that occur for instance in logistics and robotics by modeling the problem in a declarative language such as the Planning Domain Definition Language (PDDL) [2] and developing powerful planners that act as solvers for such problems. Based on an initial situation, the planner has to find a sequence of actions that lead to a specified goal when executed. The planning domain describes how the world may evolve, containing a set of fluents and a set of actions comprising preconditions and effects modeled using those fluents. The planning problem defines what objects are present, how the world looks like in the beginning and the goal that should be reached. In practice however, developing planning domains can be quite cumbersome, because there are many mistakes one can make while at the same time receiving little feedback from the planner regarding syntactical or semantical issues. There is a wide variety of planners that use different algorithms and may or may not support certain features. [1] To the domain developer, these planners might appear like a black box that often gives nothing more than “no plan found" if there is an issue or runs into a timeout in the worst case. Research in Explainable AI [3] aims to develop methods to tell the user in an easy to understand way why a complex intelligent system has made a certain decision or has failed. For example, a doctor should know why an expert system suggests a diagnosis based on the given symptoms. In a similar fashion, the user of a planner should receive an explanation why a plan couldn’t be found [4]. There are three main questions that arise when using a planner:

1. No plan can be found given a planning domain and a problem instance. It is certain that the problem instance is modeled correctly or is there an error in the planning domain?
2. Why can’t a plan be found for a problem instance given that the planning domain is modeled correctly?
3. Why did the plan execution fail?

In this paper, our goal is to find a way to answer the first of these questions. Using different algorithms, we aim on developing a simple debugger that is able to identify possible errors in a planning domain given that one or more problem instances for that domain are given where it is known that a plan exists. Subsequently we evaluate our debugger by systematically modifying domains from the International Planning Competition (IPC) and checking how many errors are found by our debugger. Finally, we discuss the obtained test results and analyze the performance of the different algorithms tested.

2 Related Work

PDDL is a declarative specification language first introduced by [2] with a LISP-like syntax that allows developers to model planning domains and problem instances. A domain consists of a set of fluents that represents the current state of the world as well as a set of

[1] https://planning.wiki/

actions that have effects on those fluents. Each action has a precondition that determines in what state that action can be executed. The problem instance contains information about which fluents hold in the initial situation and what condition must hold so that the planning goal is met. PDDL is widely accepted in the field of AI planning, as evidenced by the wide range of planners understanding that language. A planner is a program that parses domain and problem files and then tries to find a plan that meets the goal. If successful, the planner outputs the plan as a sequence of actions. Otherwise, the planner returns an error or runs out of time or computational resources. Different planners support various features and versions of PDDL and use different algorithms internally. An existing tool used to debug planning domains is the Plan Validator, developed by [5]. At first the program reads a domain file along with a plan. Then, the validator checks if the plan is executable in respect to the domain. It applies the actions given in the plan to an internal model while keeping track of the state of the fluents along with the actions that are responsible for altering each fluent. Finally, it checks if each action precondition holds and provides a detailed report containing all precondition violations as well as possible explanations for these violations. The Plan Validator can be useful when it is known in advance what the resulting plan should look like. In our scenario however, only the initial situation and the goal are known. With the goal of debugging systems too complex to understand in mind, the Delta Debugging algorithm has been proposed by [6]. It isolates failures by simplifying a test case to the bare minimum so that the failure still occurs. For example, the Delta Debugging algorithm has been used to reduce a large web page that crashes a browser down to a file containing a few HTML tags that still produce the failure. Aside from the Plan Validator, there exist other tools designed to help users in developing PDDL domains. PDDL Studio [7] is a simple IDE that helps the user to write and test PDDL domains. It provides graphical support such as syntax highlighting as well as marking syntactical and semantical errors. Another small collection of tools for the purpose of aiding PDDL developers is myPDDL, which has been developed by [8]. It contains a syntax highlighting scheme, a script to generate type diagrams from a domain as well as a tool to generate fluent assignments. There is also a PDDL plugin for the widely used editor Visual Studio Code, developed by [9]. It offers syntax highlighting, auto completion, and integration of several planners. In addition to that, it provides tools from planutils, a collection of other smaller tools for planning maintained by [10]. It could be useful to integrate our debugger into such a tool in the future.

3 Methodology

3.1 Delta Debugging

We decide to use the Delta Debugging algorithm first proposed by [11] as the basis of our implementation, because it doesn't require detailed knowledge of the system that it should debug. The input of the algorithm is a program C and a test case $test(h)$ that decides whether a hypothesis $h \subseteq C$ passes or fails. The algorithm can be applied to any input program that can be split into multiple fragments, such as lines of code. A diagnosis is obtained by repeatedly generating and testing hypotheses that are generated by systematically omitting or re-adding fragments. The algorithm runs until a set of fragments can be identified that causes the failing of the test case. An empty set of fragments is an empty program, meaning that it passes the test case, such that $test() = PASS$. The set containing all fragments is the original program that doesn't pass the test case, such that $test(C) = FAIL$.

Algorithm 1 Iterative version of the min delta debugging algorithm. The set of all fragments C is given as input. The algorithm returns the smallest subset of C that still produces the failure.

```
1: c ← C
2: n ← 2
3: while |c| ≥ n do
4:     subsets ← getSubsets(c, n)
5:     if ∃c′ ∈ subsets : test(c′) = FAIL then
6:         c ← c′
7:         n ← 2
8:     else if ∃c′ ∈ subsets : test(c \ c′) = FAIL then
9:         c ← c \ c′
10:        n ← max(n − 1, 2)
11:    else if |c| > n then
12:        n ← min(|c|, 2n)
13:    else
14:        break
15:    end if
16: end while
17: return c
```

There are two variants of the Delta Debugging algorithm that were originally described in their recursive forms. The min algorithm finds the smallest set of fragments that need to be present in order to cause a failure, as shown in Algorithm 1. It works by using the set of all fragments C as initial working set c, where $test(c) = FAIL$. The variable n holds the current level of granularity, which is 2 in the beginning. This means that at first $getSubsets(c, n)$ splits the working set into two disjoint subsets. If it is found that one of those subsets c' is sufficient to produce the failure, the hypothesis is accepted and the subset shall become the working set, with the granularity being reset to 2. Otherwise the algorithm checks whether the complement of a subset c' in respect to the current working set $c \setminus c'$ produces the failure. In that case, $c \setminus c'$ becomes the next working set and the granularity is decremented. If no subset or the complement of a subset is sufficient to produce the failure, the level of granularity is increased. The iteration stops if the cardinality of the working set becomes smaller than the granularity.
In contrast to the min algorithm, the diff algorithm keeps two working sets where $test(c_s) = PASS$ and $test(c_f) = FAIL$, as shown in Algorithm 2. It aims to minimize the difference $c_f \setminus c_s$. In the original pa-

Algorithm 2 Iterative version of the diff delta debugging algorithm. The set of all fragments C is given as input. The algorithm minimizes the difference between the failing set c_f and the passing set c_s. The difference between those sets are the faulty fragments.

$c_s \leftarrow \emptyset$
$c_f \leftarrow C$
$n \leftarrow 2$
while $|c_f \setminus c_s| \geq n$ **do**
  $diff \leftarrow c_f \setminus c_s$
  $subsets \leftarrow getSubsets(diff, n)$
  if $\exists c' \in subsets : test(c_s \cup c') = FAIL$ **then**
    $c_f \leftarrow c_s \cup c'$
    $n \leftarrow 2$
  else if $\exists c' \in subsets : test(c_f \setminus c') = PASS$ **then**
    $c_s \leftarrow c_f \setminus c'$
    $n \leftarrow 2$
  else if $\exists c' \in subsets : test(c_s \cup c') = PASS$ **then**
    $c_s \leftarrow c_s \cup c'$
    $n \leftarrow max(n-1, 2)$
  else if $\exists c' \in subsets : test(c_f \setminus c') = FAIL$ **then**
    $c_f \leftarrow c_f \setminus c'$
    $n \leftarrow max(n-1, 2)$
  else if $|diff| > n$ **then**
    $n \leftarrow min(|diff|, 2n)$
  else
    break
  end if
end while
return $c_f \setminus c_s$

per, it is referred to as the general Delta Debugging algorithm. It is able to handle cases where $test(h)$ is neither $PASS$ nor $FAIL$, which can occur when the hypothesis can't be tested due to compile errors for example. Such a scenario won't occur in our setup however, but for the sake of completeness we will test the diff algorithm nonetheless.

Additionally, we introduce a third variant of the Delta Debugging algorithm which we call the max algorithm that finds the largest set of fragments where no failure occurs. The code for the max algorithm is shown in Algorithm 3 and resembles the min algorithm in most parts. Unlike the min algorithm it accepts a hypothesis h if $test(C \setminus h) = PASS$.

The function $getSubsets(c, n)$ is used by all three algorithms and splits the set c into n disjoint subsets of roughly equal size. For example, $getSubsets(\{0,1,2,3,4\}, 3) = \{\{0,1\},\{2,3\},\{4\}\}$.

3.2 Why planning fails

There exist multiple causes why a planner can't obtain a plan for a domain and problem instance. The most common issue is that the domain is correct, but the problem is either not solvable or too complex, resulting in a timeout. Using a different planner can also have a significant impact on the result, because they use different search algorithms with specific heuristics. Within a domain there may be multiple sources of errors, such as missing action effects, incorrect preconditions or the domain being abstracted wrongly. For example, the following excerpt from a PDDL domain contains an error in an action precondition, so that no plan can be found even if the problem instances is solvable. In the domain, a robot can transport items between several places. The robot can carry at most one item at once. Upon closer inspection, it becomes obvious that the line `(holding ?i) ; p3` must be deleted to fix the domain, because the robot can't be empty and hold an item at the same time.

Algorithm 3 Iterative version of the max delta debugging algorithm. The set of all fragments C is given as input. The algorithm returns the largest subset of C that doesn't fail. Parts that differ from the min algorithm are marked red.

$c \leftarrow C$
$n \leftarrow 2$
while $|c| \geq n$ **do**
  $subsets \leftarrow getSubsets(c, n)$
  if $\exists c' \in subsets : test(C \setminus c') = PASS$ **then**
    $c \leftarrow c'$
    $n \leftarrow 2$
  else if $\exists c' \in subsets : test(C \setminus (c \setminus c')) = PASS$ **then**
    $c \leftarrow c \setminus c'$
    $n \leftarrow max(n-1, 2)$
  else if $|c| > n$ **then**
    $n \leftarrow min(|c|, 2n)$
  else
    break
  end if
end while
return c

```
(:action take
  :parameters (?p - Place ?i - Item)
  :precondition (and
    (robot-at ?p)     ; p0
    (robot-empty)     ; p1
    (item-at ?p ?i)   ; p2
    (holding ?i)      ; p3
  )
  :effect (and
    (holding ?i)
    (not (robot-empty))
    (not (item-at ?p ?i))
  )
)

(:action put
  :parameters (?p - Place ?i - Item)
  :precondition (and
    (robot-at ?p) ; p4
    (holding ?i)  ; p5
  )
  :effect (and
    (not (holding ?i))
    (robot-empty)
    (item-at ?p ?i)
  )
)
```

3.3 Use in Plan Debugging

In our scenario, an erroneous planning domain is given along with one or more correct solvable problem in-

stances. We want to obtain a diagnosis without any additional hints or assistance by the user. As of now, we have decided to limit our work to finding faults in preconditions, because removing preconditions leads to a relaxation of the planning domain, increasing the chance that a plan can be found. Finding errors in effects is harder because the debugger would need a good understanding of the domain to identify which effects are needed to reach the planning goal. As shown in the PDDL code above, an action precondition P^a can be broken down into the conjunction $P_0^a \wedge P_1^a \wedge \ldots \wedge P_n^a$. If the precondition takes another form, such as a disjunction or a single predicate, then $P^a = P_0^a$. From now on, we will refer to a single P_n^a as a precondition. In the code above, the preconditions are $P^{take} = p0 \wedge p1 \wedge p2 \wedge p3$ and $P^{put} = p4 \wedge p5$, meaning that the two actions have 6 preconditions in total. The Delta Debugging algorithm comes in handy when debugging action preconditions, because we can set $C = \{p0, p1, p2, p3, p4, p5\}$. Removing elements from the set results in preconditions being deleted from the domain. To the algorithm, it doesn't matter how the preconditions are distributed among actions, because it uses the full set of all preconditions as input. For example, the hypothesis $\{p0, p1, p4\}$ gives the following modified domain:

```
(:action take
  :parameters (?p - Place ?i - Item)
  :precondition (and
    (robot-at ?p) ; p0
    (robot-empty) ; p1
  )
  :effect (and
    (holding ?i)
    (not (robot-empty))
    (not (item-at ?p ?i))
  )
)

(:action put
  :parameters (?p - Place ?i - Item)
  :precondition (and
    (robot-at ?p) ; p4
  )
  :effect (and
    (not (holding ?i))
    (robot-empty)
    (item-at ?p ?i)
  )
)
```

This way the debugger identifies one or more preconditions that may contain an error. In our example, the final diagnosis should be $\{p3\}$.

3.4 Implementation

We develop a prototype that takes an erroneous PDDL domain file and a set of correct problem files as input. The type of Delta Debugging algorithm to be used, the planner to be called along with the timeout can also be configured. As shown in Figure 1, the debugger starts by parsing the domain and then extracting the preconditions. The structure of the domain is stored in a simple syntax tree that holds actions and other definitions such as fluents or constants. Each precondition gets assigned a unique ID for further processing. The recompiler takes the parsed domain file and a set of preconditions to include as input and generates a new, temporary domain file. Before calling the Delta Debugging algorithm, the debugger tests each problem file against the original domain. All problem files where a plan is found are ignored. If all problem files pass, we assume there is no error and the debugger exits. Additionally, the debugger generates a temporary domain file with no preconditions at all so that $h = \emptyset$ and then calls the planner on each problem file. If there is a problem file where no plan is found, it is highly likely that action effects are incorrect and the debugger exits. Otherwise the specified version of the Delta Debugging algorithm is called. The procedure $test(h)$ is passed to the Delta Debugging algorithm and executes the following steps:

1. Obtain a new hypothesis $h = \{p0, p1, \ldots, pn\}$ from the Delta Debugging algorithm.
2. Call the recompiler to generate a temporary domain file containing only the preconditions according to the hypothesis.
3. Call the planner. If a plan can be found, the test passes. If no plan can be found or the planner runtime exceeds the given timeout, the test fails.

In case there are multiple problem files, the Delta Debugging algorithm is called once for each problem file, resulting in multiple diagnoses, with each diagnosis consisting of one or more errors. Since each diagnosis might contain some false positives, returning the union of all diagnoses would even further increase the amount of false positives. Our line of reasoning is that errors that are contained in multiple diagnoses are more likely to be true positives. Hence, our debugger counts how often each error is reported. Only the 3 most reported errors are then presented to the user. Algorithm 4 gives a broad overview of the control flow within our implementation. The debugger is written in Java. When called, different arguments can be set, like (1) the algorithm to be used (`min`, `max` or `diff`), (2) the timeout for the planner, and the paths (2) to the domain file and (3) to the problem folder.

4 Evaluation

4.1 Setup

In order to evaluate how well our implementation performs, we use a set of selected working planning domains, introduce errors in those domains, and then check whether our implementation correctly identifies those errors. Instead of coming up with new domains, we decide to use existing domains from the International Planning Competition (IPC) [12], as they reflect typical planning scenarios. The IPC is a recurring event where developers can benchmark their planners and compete with each other. A large repository is

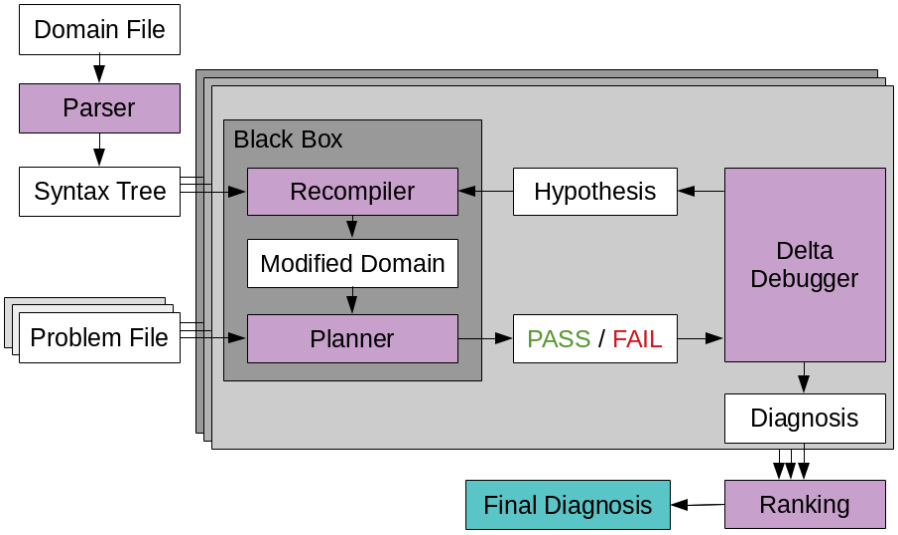


Figure 1: An overview showing the components of our implementation (purple) as well as the data exchanged between them (white).

Algorithm 4 Proposed Implementation.

```
1: procedure MAIN(domainFile, problemFiles, algorithm)
2:     syntaxTree ← parseDomain(domainFile)
3:     allPreconds ← syntaxTree.preconds
4:     for problemFile ∈ problemFiles do
5:         if test(problemFile, allPreconds) = PASS then
6:             remove(problemFiles, problemFile)
7:         end if
8:         if test(problemFile, {}) = FAIL then
9:             return OTHER_ERROR
10:        end if
11:    end for
12:    if is_empty(problemFile) then
13:        return NO_ERROR
14:    end if
15:    diagnosedCount ← {}
16:    for precond ∈ allPreconds do
17:        append(diagnosedCount, ⟨precond, 0⟩)
18:    end for
19:    for problemFile ∈ problemFiles do
20:        currentProblemFile ← problemFile
21:        diag ← runDelta(allPreconds, algorithm)
22:        for precond ∈ diag do
23:            incrementBy1(diagnosedCount, precond)
24:        end for
25:    end for
26:    sortDescending(diagnosedCount)
27:    return getFirst3Preconds(diagnosedCount)
28: end procedure
29: procedure TEST(problemFile, h)
30:    compiledDomain ← recompile(syntaxTree, h)
31:    writeFile(temp, compiledDomain)
32:    result ← callPlanner(temp, problemFile)
33:    if result = TIMEOUT then
34:        return FAIL
35:    end if
36:    return result
37: end procedure
```

maintained where hundreds of domains from previous competitions are archived. Each domain is accompanied by up to a few dozen problem instances. Furthermore, we decide to use the Optic planner as the main planner in our evaluation, because it performs well and supports a wide range of features [13]. In order to narrow down the number of domains, we developed a script that calls the Optic planner on each domain and discards all domains that contain unsupported features or where obtaining a solution takes more than 10 seconds on our machine equipped with an AMD Ryzen 9 5950X and 32GB RAM. In addition to the domains from the IPC, we also include a crafting domain in our evaluation, since it contains some interesting mechanics such as durative actions and functional fluents. It was designed by us for teaching purposes and simulates a simple manufacturing process. This gives a total of 10 domains to be evaluated, described in Table 1. The covered scenarios comprise navigation in grids, controlling rovers and satellites, logistics, or board games. In our evaluation, we consider 5 different types of errors that can occur in preconditions of planning domains, as listed in Table 2. These errors break the domain so that the planner is unable to obtain a plan for some of the problem instances. It is still possible that the fault does not affect the solving of a simpler problem that does not utilize all actions. For example, consider the following PDDL action:

```
(:action sleep
  :parameters (?p - Place)
  :precondition (and
    (is-tired) ; p0
    (at BED)   ; p1
  )
  :effect (not (is-tired))
)
```

Domain	# Preconditions	# Actions	# Instances
1998-grid-round-2-strips	30	5	3
1998-gripper-round-1-adl	6	3	16
1998-mystery-prime-round-1-strips	29	4	16
1998-logistics-round-1-strips	31	6	16
2000-elevator-strips-simple-typed	9	4	16
2002-satellite-strips-automatic	16	5	13
2004-pipesworld-no-tankage-nontemporal-strips	40	6	16
2006-rovers-propositional	45	9	16
2011-peg-solitaire-sequential-satisficing	11	3	16
2021-crafting	16	5	7

Table 1: The names of the domains used for the evaluation, along with the total number of preconditions in the domain, the number of actions and accompanying problem instances.

Abbreviation	Description
A	Adding a new precondition that breaks the domain.
AA	Adding two new preconditions that both independently break the domain. Those two preconditions are either in the same action or distributed across two actions.
M	Modifying an existing precondition, resulting in a broken domain.
MM	Modifying two existing preconditions that both independently break the domain. Either one action may be affected by two modifications or two actions by one modification each.
AM	Adding a new precondition and modifying an existing precondition, both breaking the domain independently.

Table 2: The types of errors introducted in each domain for the evaluation.

If we add a precondition that might break the domain:

```
(:action sleep
  :parameters (?p - Place)
  :precondition (and
    (is-tired) ; p0
    (at BED)   ; p1
    (at COUCH) ; p2
  )
  :effect (not (is-tired))
)
```

Assuming that the agent can not be at two places at the same time, it becomes impossible to execute this action. It should be mentioned that the debugger might as well identity `p1` as the erroneous precondition since removing it would also resolve the conflict. Modifying `p0` might not make the action itself impossible to execute, but might prevent it from being executed in the right situation:

```
(:action sleep
  :parameters (?p - Place)
  :precondition (and
    (not (is-tired)) ; p0
    (at BED)         ; p1
  )
  :effect (not (is-tired))
)
```

For each of the 10 domains and 5 error types, we create 3 different testcases, each consisting of the domain with an error matching the error type. Each testcase is then solved with the three variants of the Delta Debugging algorithm (min, max and diff) with a timeout of 1 second or 10 seconds, resulting in a total of 900 test runs. The execution of the tests is fully automated using a script. We decide to use a timeout of 10 seconds, because for the domains we have selected for this evaluation, the planner usually finds a plan within that time. Additionally we want to run our tests with a timeout of 1 second, because we expect that it will increase the rate of timeouts and will thus negatively affect the quality of the results. The test script logs the total number of planner runs r and how often the planner fails. The planner can fail because either a timeout occurs (counted as c_{to}) or it reports that the problem is unsolvable (counted as c_u). Among the reported erroneous preconditions, the script documents the number of true positives, false positives and false negatives.

4.2 Results

Since the full dataset of test results contains 900 rows, we decide to summarize the results within a few tables and compare the performance of the different algorithms, timeouts and error types. Each domain has a set of preconditions P and the subset P_e of faulty preconditions with $n = |P_e|$. Our debugger returns a diagnosis, which is the set P_d. Each table reports the following cumulative results:

- $t_\mu[s]$ - The average execution time per testcase in seconds for the specified set of testcases.
- $t_\sigma[s]$ - The standard deviation of the execution time per testcase in seconds for the specified set of testcases.
- r_μ - The average number of planner runs per testcase and problem instance.
- $d_1[\%]$ - The percentage of testcases where $P_e \cap P_d \neq \emptyset$, i.e. the diagnosis contains at least one

Algorithm	Timeout	$t_\mu[s]$	$t_\sigma[s]$	r_μ	$d_1[\%]$	$d_n[\%]$	fp_μ	$to[\%]$
MIN	1	7	4	17	57	45	2.16	55
MIN	10	77	54	18	57	45	2.16	53
MAX	1	31	41	18	78	62	1.8	39
MAX	10	150	210	15	82	64	1.76	33
DIFF	1	6	3	6	51	40	2.21	53
DIFF	10	58	36	7	64	49	2.01	48

Table 3: Comparing the different algorithms with two different timeouts.

incorrect precondition.

- $d_n[\%]$ - The percentage of testcases where $P_e \subseteq P_d$, i.e. all incorrect preconditions are contained in the diagnosis.
- fp_μ - The average value of $|P_d \setminus P_e|$, i.e. the number of falsely reported correct preconditions.
- $to[\%]$ - The average value of $\frac{c_{to}}{c_{to}+c_u}$, i.e. the planner fails due to timeout.

As evident from the results in Table 3, the max algorithm has the highest chance to correctly identify faulty preconditions, correctly diagnosing at least one error in 82% of testcases. The min algorithm in contrast does the worst at 57%, with the diff algorithm doing slightly better at 64%. Both the min and max algorithm need about the same number of planner calls, while the diff algorithm does less than half and requires the least runtime as a consequence. However, there is a strong difference in runtime between the min and max algorithm. This is because the min algorithm attempts to construct a domain as simple as possible that produces the failure, while the max algorithm builds a domain as complex as possible that does not fail. It is reasonable that more complex domains typically require more computational power, effectively increasing the runtime as well as the number of timeouts. $t_\sigma[s]$ in relation to $t_\mu[s]$ is fairly high, meaning that the runtime of the debugger depends mostly on the complexity of each individual testcase. Lowering the timeout from $10s$ to $1s$ has a relatively low impact on the quality of the results, while drastically improving the runtime of the debugger. As expected, the number of timeouts is higher when the timeout is lower. In our case, a timeout of $1s$ still gives good results because the domains and problems chosen for this evaluation can mostly be solved in less than a second. The max algorithm has the lowest number of timeouts among failures. As we aim to use the debugger online while developing domains, response time of the debugger matters.

The chance of finding at least one error is higher when two errors are present, as shown in Table 4. In the min and diff algorithms, the runtime and number of planner calls do not show much variation depending on the error type. The max algorithm however makes less than half of calls when there is one error instead of two. When considering just the max algorithm, testcases with additional preconditions take longer processing time than ones with modified preconditions. This is because each additional precondition increases the number of total preconditions, thus increasing the number of combinations the Delta Debugging algorithm has to test. The error types AA, MM and AM require a higher number of planner runs than A and M, because unlike the min or diff algorithms, the max algorithm attempts to find all errors present instead of stopping when one error is found. Furthermore, it seems easier to identify additional preconditions than modified ones.

When comparing the different domains in Table 5, we find that the max algorithm gives the best results in all of them. Other than that, the diff algorithm performs better than the min algorithm in most domains. There is plenty of variation regarding the runtime among domains, which can be explained by the varying complexity of the domains. The simplest domain is arguably 1998-griper-round1-adl with only 6 preconditions, resulting in an average runtime of 23 seconds and a very low timeout rate. d_n is highest among all domains and fp_μ is the lowest. A more challenging domain is 1998-mystery-prime-round-1-strips, taking an average of 117 seconds and having a high timeout rate. Consequently, d_1 and d_n are lower while fp_μ is high. The number of planner calls is well below the number of tries it would take to find an error by brute force in all domains.

5 Conclusion

Our goal was to debug planning domains with minimum user interaction. In our use case, the domain developer provides a broken planning domain along with one or more problem instances to the debugger. We started by discussing the problems surrounding this task and limited the scope of our research to finding errors in the action preconditions. Then we explained how planning domains can be debugged using the Delta Debugging algorithm. In addition to the two already existing variants of this algorithm, min and diff, we proposed a third variant called max. Subsequently we developed a prototype that implements all three variants of the Delta Debugging algorithm and can interact with a planner. It systematically alters the planning domain until one or more faulty preconditions can be identified. After that, we performed a large number of tests where we introduced different kinds of errors to publicly available domains and checked how well our debugger can find those errors. The max algorithm proved to give the best results, finding at least one error in 82% of the test cases and

Algorithm	Error Type	$t_\mu[s]$	$t_\sigma[s]$	r_μ	$d_1[\%]$	$d_n[\%]$	fp_μ	$to[\%]$
MIN	A	83	58	21	56	56	2.16	62
MIN	AA	77	46	18	70	40	2.16	54
MIN	M	67	50	17	50	50	2.23	53
MIN	MM	76	59	15	53	35	2.1	47
MIN	AM	83	56	18	60	43	2.13	52
MAX	A	98	97	8	80	80	1.73	43
MAX	AA	212	267	21	96	60	1.8	29
MAX	M	87	144	8	70	70	1.9	35
MAX	MM	165	212	20	80	55	1.6	26
MAX	AM	189	260	20	86	58	1.76	30
DIFF	A	57	35	6	50	50	2.23	55
DIFF	AA	67	38	7	66	40	2.1	48
DIFF	M	53	34	6	60	60	2.06	49
DIFF	MM	56	39	6	73	46	1.8	41
DIFF	AM	60	36	7	73	48	1.86	47

Table 4: Comparing the different algorithms and error types. Only results with a timeout of $10s$ are shown.

Algorithm	Domain	$t_\mu[s]$	$t_\sigma[s]$	r_μ	$d_1[\%]$	$d_n[\%]$	fp_μ	$to[\%]$
MIN	1998-grid-round-2-strips	86	67	9	60	40	2.0	77
MAX	1998-grid-round-2-strips	107	152	22	80	53	1.66	45
DIFF	1998-grid-round-2-strips	56	17	7	26	20	2.06	71
MIN	1998-gripper-round-1-adl	8	13	10	100	96	0.66	6
MAX	1998-gripper-round-1-adl	23	41	11	100	93	0.73	10
DIFF	1998-gripper-round-1-adl	7	14	5	100	90	0.66	5
MIN	1998-logistics-round-1-strips	117	12	7	6	3	2.93	82
MAX	1998-logistics-round-1-strips	510	385	25	53	46	2.26	42
DIFF	1998-logistics-round-1-strips	109	6	7	26	16	2.73	78
MIN	1998-mystery-prime-round-1-strips	94	24	4	6	3	2.93	92
MAX	1998-mystery-prime-round-1-strips	225	117	11	60	40	2.33	78
DIFF	1998-mystery-prime-round-1-strips	93	23	4	20	13	2.73	92
MIN	2000-elevator-strips-simple-typed	39	3	26	100	73	1.93	18
MAX	2000-elevator-strips-simple-typed	24	20	9	100	73	1.6	5
DIFF	2000-elevator-strips-simple-typed	36	8	7	100	76	1.8	22
MIN	2002-satellite-strips-automatic	50	28	19	100	80	1.8	35
MAX	2002-satellite-strips-automatic	106	92	25	100	86	1.6	17
DIFF	2002-satellite-strips-automatic	46	11	7	93	76	1.8	32
MIN	2004-pipesworld-no-tankage-strips	143	57	13	26	16	2.73	64
MAX	2004-pipesworld-no-tankage-strips	207	236	16	80	56	1.93	26
DIFF	2004-pipesworld-no-tankage-strips	67	18	8	80	60	2.13	34
MIN	2006-rovers-propositional	101	56	24	86	66	1.8	53
MAX	2006-rovers-propositional	86	58	16	100	76	1.53	18
DIFF	2006-rovers-propositional	71	33	8	73	50	2.0	44
MIN	2011-peg-solitaire-satisficing	92	19	4	20	13	2.66	87
MAX	2011-peg-solitaire-satisficing	184	72	9	60	43	2.26	72
DIFF	2011-peg-solitaire-satisficing	87	16	4	33	20	2.46	86
MIN	2021-crafting	43	37	63	73	56	2.13	20
MAX	2021-crafting	30	53	9	93	76	1.66	16
DIFF	2021-crafting	11	14	9	93	66	1.73	13

Table 5: Comparing the different domains tested. Only results of testcases with a timeout of $10s$ are shown.

all errors in 64% of test cases. However, the max algorithm shows the highest runtime among the three tested algorithms. The diff algorithm takes the least time, but gives less accurate results. In contrast, the min algorithm lies in between those two regarding runtime, but gives the worst results. Overall, the performance of our debugger is largely influenced by the structure and complexity of the domain.

There exist a number of limitation in this approach because we currently focus only on errors in preconditions but not in effects. Moreover, we are not able to deal with missing preconditions or wrong preconditions that relax the planning domain in a way that invalid or unwanted plans are found. The reason is that we use only a test if a plan can be generated if one exists, but we do not check if the plan is suitable. Thus, in future work we aim on improving the debuggers to be able to find a wider set of errors in preconditions and effects using mutation testing and providing suggestions on how to fix a domain. Furthermore, it needs to be investigate how information on expected plans can be incorporated into the approach.

References

[1] Malik Ghallab, Dana Nau, and Paolo Traverso. *Automated Planning: Theory and Practice*. The Morgan Kaufmann Series in Artificial Intelligence. Morgan Kaufmann, Amsterdam, 2004. 0

[2] Malik Ghallab, Adele Howe, Craig Knockblock, Drew McDermott, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins. PDDL The Planning Domain Definition Language. Technical report, Yale Center for Computational Vision and Control, 1998. 0

[3] David Gunning, Mark Stefik, Jaesik Choi, Timothy Miller, Simone Stumpf, and Zhong-Guang Yang. XAI - Explainable Artificial Intelligence. *Science Robotics*, 4(37), 2019. 0

[4] Maria Fox, Derek Long, and Daniele Magazzeni. Explainable planning. *ArXiv*, abs/1709.10256, 2017. 0

[5] Richard Howey and Derek Long. VAL's progress: the automatic validation tool for PDDL2.1 used in the International planning competition. In *Proceedings of the 13th International Conference on Automated Planning and Scheduling*, pages 28–37. AAAI Press, 2003. 1

[6] Andreas Zeller. Yesterday, my Program Worked. Today, it Does Not. Why? In *Software Engineering — ESEC/FSE '99*, pages 253–267. Springer Berlin Heidelberg, 1999. 1

[7] Tomas Plch, Miroslav Chomut, Cyril Brom, and Roman Barták. Inspect, Edit and Debug PDDL Documents: Simply and Efficiently with PDDL Studio. In *Proceedings of the 22nd International Conference on Automated Planning and Scheduling*. AAAI Press, 2012. 1

[8] Volker Strobel and Alexandra Kirsch. Planning in the Wild: Modeling Tools for PDDL. In *KI 2014: Advances in Artificial Intelligence*, pages 273–284. Springer International Publishing, 2014. 1

[9] Jan Dolejsi. VS Code Planning Domain Description Language Support. https://github.com/jan-dolejsi/vscode-pddl, 2022. 1

[10] Christian Muise, Florian Pommerening, Jendrik Seipp, and Michael Katz. PLANUTILS: Bringing Planning to the Masses. In *International Conference on Automated Planning and Scheduling: System Demonstrations*. 2022. 1

[11] Andreas Zeller and Ralf Hildebrandt. Simplifying and isolating failure-inducing input. *IEEE Transactions on Software Engineering*, 28(2):183–200, 2002. 1

[12] Patrick Lühne. PDDL Benchmark Instances. https://github.com/potassco/pddl-instances, 2017. 3

[13] J. Benton, Amanda Coles, and Andrew Coles. Temporal Planning with Preferences and Time-Dependent Continuous Costs. In *Proceedings of the 22nd International Conference on Automated Planning and Scheduling*, pages 2–10. AAAI Press, 2012. 4